Theory Assignment – 3

Subject: JavaScript

1. What is the difference between Frontend and Backend JavaScript?

Frontend and Backend JavaScript are two different ways of using JavaScript in web development.

Frontend JavaScript:

- Frontend JavaScript runs in the user's web browser.
- It is used to make websites interactive and attractive.
- It handles buttons, forms, animations, menus, and user actions.
- Example: When a user clicks a button, JavaScript displays a message.

Backend JavaScript:

- Backend JavaScript runs on the server using environments like Node.js.
- It is used to handle server-side operations.
- It manages databases, user authentication, and server requests.
- Example: When a user logs in, backend JavaScript checks the username and password.

Main Difference:

Frontend JavaScript	Backend JavaScript
Runs in the browser.	Runs on the server.
Handles user interface.	Handles server operations.
Manages user interactions.	Manages databases and APIs.
Example: React.js	Example: Node.js

---

2. What is DOM?

DOM stands for **Document Object Model**.

The DOM is a programming interface that represents an HTML document in the form of a tree structure. It allows JavaScript to access, change, add, or delete HTML elements and their content.

For example, JavaScript can change the text, color, and style of a webpage using the DOM.

Example:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2 id="demo">Hello World</h2>

<button onclick="changeText()">Click Me</button>

<script>
function changeText() {
    document.getElementById("demo").innerHTML =
    "Welcome to JavaScript";
}
</script>

</body>
</html>
```

Explanation:

- `document` represents the HTML page.
- `getElementById()` selects an element using its ID.
- `innerHTML` changes the content of the selected element.

Thus, DOM helps JavaScript interact with HTML elements.

---

3. What is an Event in JavaScript?

An event is an action or occurrence that happens in a webpage and can be detected by JavaScript.

Events are mainly used to make websites interactive.

Common JavaScript Events:

1. `click` – Occurs when the user clicks an element.
2. `mouseover` – Occurs when the mouse moves over an element.
3. `keydown` – Occurs when a keyboard key is pressed.
4. `submit` – Occurs when a form is submitted.
5. `load` – Occurs when a webpage or resource finishes loading.

Example:

```
<button onclick="showMessage()">Click Me</button>

<script>
function showMessage() {
    alert("Button clicked successfully!");
}
</script>
```

Explanation: When the user clicks the button, the click event occurs and JavaScript displays an alert message.

---

4. What is Event Loop in JavaScript?

The Event Loop is a mechanism in JavaScript that manages the execution of code and helps JavaScript handle asynchronous operations.

JavaScript is single-threaded, which means it executes one main task at a time.

The Event Loop checks whether the Call Stack is empty and then moves pending callback functions from the appropriate queue to the Call Stack.

Main Components:

1. **Call Stack:** Executes JavaScript functions.
2. **Web APIs:** Handle operations like timers and browser events.
3. **Callback Queue:** Stores callbacks waiting to execute.
4. **Microtask Queue:** Stores higher-priority tasks like Promise callbacks.
5. **Event Loop:** Coordinates the execution of pending tasks.

Example:

```
console.log("Start");

setTimeout(function() {
    console.log("Hello");
}, 2000);

console.log("End");
```

Output:

```
Start
End
Hello
```

Explanation:

- First, "Start" is printed.
- Then `setTimeout()` schedules a callback.
- "End" is printed without waiting for the timer.
- After approximately 2 seconds, "Hello" is printed.

Thus, the Event Loop helps JavaScript manage asynchronous operations.

---

5. What is Hoisting in JavaScript?

Hoisting is a JavaScript behavior in which variable and function declarations are processed before the code is executed.

It allows certain variables and functions to be accessed before their declaration appears in the code.

Example 1: Using var

```
console.log(a);

var a = 10;
```

Output:

```
undefined
```

Explanation: The declaration of `a` is hoisted, but its value is assigned later. Therefore, the output is `undefined`.

Example 2: Function Hoisting

```
greet();

function greet() {
    console.log("Hello World");
}
```

Output:

```
Hello World
```

Explanation: Function declarations are hoisted, so they can be called before their declaration.

Note:

- `var` is hoisted and initialized with `undefined`.
- `let` and `const` are hoisted but cannot be accessed before initialization because of the Temporal Dead Zone (TDZ).

---

6. What is Event Bubbling?

Event bubbling is a process in JavaScript in which an event starts from the target element and then moves upward through its parent elements.

For example, if a button is inside a div, clicking the button can trigger the button's event first and then the div's event.

Example:

```
<div id="parent">
    <button id="child">Click Me</button>
</div>
```

```
<script>
document.getElementById("parent")
.addEventListener("click", function() {
    alert("Parent clicked");
});

document.getElementById("child")
.addEventListener("click", function() {
    alert("Button clicked");
});
</script>
```

Explanation:

When the button is clicked:

1. The button's event executes first.
2. The event then moves to the parent div.
3. The parent's event executes.

This process is called Event Bubbling.

We can stop event bubbling by using:

```
event.stopPropagation();
```

---

7. What is the difference between Named Function, Anonymous Function and Arrow Function?

JavaScript provides different ways to create functions.

1. Named Function:

A named function is a function that has a specific name. It can be called using that name.

Example:

```
function add() {
    console.log("Addition");
}

add();
```

2. Anonymous Function:

An anonymous function is a function that does not have its own name. It is commonly assigned to a variable or passed as a callback.

Example:

```
let greet = function() {
```

```
    console.log("Hello World");
};

greet();
```

3. Arrow Function:

An arrow function is a shorter way of writing a function. It uses the => symbol.

Example:

```
let greet = () => {
    console.log("Hello World");
};

greet();
```

Main Difference:

Named Function	Anonymous Function	Arrow Function
Has a function name.	Has no function name.	Uses => syntax.
Can be called by its name.	Usually stored in a variable or passed as a callback.	Usually stored in a variable or passed as a callback.
Has its own `this` based on how it is called.	Has its own `this` based on how it is called.	Does not have its own `this`.
Uses `function` keyword.	Uses `function` keyword.	Uses arrow syntax.

---

8. Explain for-of and for-in Loop.

JavaScript provides `for-of` and `for-in` loops to iterate over data.

1. for-of Loop:

The `for-of` loop is used to iterate over the values of iterable objects such as arrays and strings.

Example:

```
let fruits = ["Apple", "Mango", "Banana"];

for (let fruit of fruits) {
    console.log(fruit);
}
```

Output:

```
Apple
Mango
Banana
```

Explanation: The `for-of` loop accesses each value of the array one by one.

2. for-in Loop:

The `for-in` loop is used to iterate over the enumerable property keys of an object. It can also be used with arrays to access their indexes.

Example:

```
let student = {
    name: "Rahul",
    age: 20,
    course: "BCA"
};

for (let key in student) {
    console.log(key, student[key]);
}
```

Output:

```
name Rahul
age 20
course BCA
```

Explanation: The `for-in` loop accesses the property names of the object.

Difference:

for-of Loop	for-in Loop
Used to get values.	Used to get property keys.
Works with iterable objects.	Works with enumerable object properties.
Commonly used with arrays and strings.	Commonly used with objects.
Returns array elements.	Returns array indexes when used on arrays.

---

9. Explain Any Five Array Methods in JavaScript.

Array methods are built-in JavaScript functions used to perform different operations on arrays.

Here are five commonly used array methods:

1. push()

The `push()` method adds one or more elements at the end of an array.

Example:

```
let fruits = ["Apple", "Mango"];
```

```
fruits.push("Banana");

console.log(fruits);
```

Output:

```
["Apple", "Mango", "Banana"]
```

2. pop()

The `pop()` method removes the last element from an array.

Example:

```
let fruits = ["Apple", "Mango", "Banana"];

fruits.pop();

console.log(fruits);
```

Output:

```
["Apple", "Mango"]
```

3. shift()

The `shift()` method removes the first element from an array.

Example:

```
let numbers = [10, 20, 30];

numbers.shift();

console.log(numbers);
```

Output:

```
[20, 30]
```

4. unshift()

The `unshift()` method adds one or more elements at the beginning of an array.

Example:

```
let numbers = [20, 30];

numbers.unshift(10);

console.log(numbers);
```

Output:

```
[10, 20, 30]
```

5. map()

The `map()` method creates a new array by applying a function to every element of the original array.

Example:

```
let numbers = [1, 2, 3, 4];

let result = numbers.map(function(num) {
    return num * 2;
});

console.log(result);
```

Output:

```
[2, 4, 6, 8]
```

Explanation: The `map()` method multiplies every element by 2 and returns a new array.

---

10. What is Callback Method?

A callback is a function that is passed as an argument to another function and is executed later by that function.

Callbacks are commonly used in JavaScript for handling events and asynchronous operations.

Example:

```
function greet(name, callback) {
    console.log("Hello " + name);

    callback();
}

function message() {
    console.log("Welcome to JavaScript!");
}

greet("Rahul", message);
```

Output:

```
Hello Rahul
Welcome to JavaScript!
```

Explanation:

1. The `greet()` function accepts two arguments: `name` and `callback`.

2. The `message()` function is passed as a callback.
3. First, the greeting message is printed.
4. Then the callback function is executed.

Therefore, a callback is a function passed to another function to be executed when required.

Conclusion:

Callbacks help JavaScript execute functions in a flexible way and are useful for managing events and asynchronous tasks.